DevOps Infrastructure – Unit 1 Notes (Part 3)

Control Groups (cgroups), Container Images & Layers

---

1. Control Groups (cgroups)

Definition:

Control Groups (cgroups) are a Linux kernel feature used to:

- Limit
- Monitor
- Control

the **resource usage** of processes (including containers).

Why cgroups are Important

Without cgroups:

- One container could consume all CPU or memory
- System becomes unstable

With cgroups:

- Fair resource distribution
- Better performance control
- Prevents resource starvation

Resources Controlled by cgroups

CPU

- Limit how much CPU a container can use

Memory

- Set maximum RAM usage
- Prevent memory leaks from crashing system

Disk I/O

- Control read/write speed

Network Bandwidth

- Limit network usage (in advanced setups)

Example

If you run:

docker run -m 512m my_app

This means:

- Container can use **maximum 512 MB RAM**

How cgroups Work

1. OS creates a control group
2. Assigns processes (container) to that group
3. Applies resource limits
4. Continuously monitors usage

---

cgroups + Namespaces Together

Feature	**Role**
Namespaces	Isolation
cgroups	Resource control

Together they form the **foundation of containers**

---

2. Container Images

Definition:

A **container image** is a **read-only template** used to create containers.

It contains:

- Application code
- Dependencies

- Libraries
- Environment setup

Key Properties

- Immutable (cannot be changed once created)
- Portable (can run anywhere)
- Version-controlled

Example

A Node.js image may include:

- Node runtime
- App code
- Required packages

Image vs Container

Image	Container
Blueprint	Running instance
Static	Dynamic
Read-only	Read-write

---

3. Container Image Layers

Concept of Layers

Container images are built using **layers**.

Each instruction in a build file (like Dockerfile) creates a new layer.

Example Dockerfile

FROM ubuntu

RUN apt-get update

RUN apt-get install python3

COPY app.py /app/

This creates multiple layers:

1. Base OS (Ubuntu)
2. Update packages
3. Install Python
4. Copy application

Why Layers are Important

Efficiency

- Reuse existing layers
- Faster builds

Caching

- Only changed layers are rebuilt

Reduced Storage

- Shared layers across containers

Layer Sharing Example

If 5 containers use:

- Same Ubuntu base image

Only stored once in system

---

4. Copy-on-Write Mechanism

Concept:

When a container runs:

- It adds a **writable layer** on top of image layers

Behavior:

- Read → from image layers
- Write → to container layer

Advantage:

- Original image remains unchanged
- Multiple containers can share same base image

---

5. Image Creation Workflow

1. Developer writes Dockerfile
2. Build command creates image
3. Image stored locally or in registry
4. Container runtime uses image to run container

---

6. Summary

- cgroups control resource usage (CPU, memory, etc.)
- Images are read-only templates for containers
- Images are built using layers
- Layers improve performance, storage, and reuse
- Copy-on-write enables efficient container execution